

PRE-CLASS READING 3 OF 5

Intro to Large Language Models

Andrej Karpathy (2023). Talk + slides. Published 23 November 2023.

ASSIGNED READING

Full talk — or slides only if short on time

~1 hour

Slides only (skim)

~20–30 min

≈ 1 hour (video) or ~30 min (slides only)

Note on currency. This talk is from November 2023 — before GPT-4o, Claude 3+, and the reasoning-model wave. The specific numbers (Llama 2 70B's size, that year's Arena rankings) are dated. The *framework* — two files, three training stages, scaling laws, LLM-as-OS, the security taxonomy — is still exactly how the field teaches this material in 2026. Read it for the mental model, not the numbers. Reading 2 (AI Index 2026) gives you the current numbers for the same categories.

KEY TAKEAWAY

An LLM reduces to two files — a giant list of numbers (parameters) and a small program that runs them (~500 lines of C, no internet required). Everything else — the cost, the capability, the danger — comes from how those parameters were produced and shaped, not from running the model itself.

WHY IT MATTERS FOR THIS COURSE

This is your technical floor under Mollick's frame. Where Mollick tells you the frontier is jagged, Karpathy shows you the mechanism that makes it jagged: next-token prediction, not reasoning. Where Reading 1 asks you to test the boundary empirically, this talk explains why testing is necessary — the parameters are, in Karpathy's word, “mostly inscrutable.” Directly feeds **A1** (you're about to build with exactly this two-stage architecture) and **Class 3** (prompting only works because of what fine-tuning did in stage two).

MAIN POINTS

What an LLM actually is

- **Two files.** A parameters file (weights — for Llama 2 70B, 140GB, because each of 70 billion parameters is a 2-byte float16 number) and a run file (~500 lines of C, no dependencies). Compiled, this pair needs no internet connection to operate. The complexity is entirely in producing the parameters, not in running them.
- **Training = lossy compression of the internet.** Llama 2 70B: ~10TB of crawled text, 6,000 GPUs, 12 days, ~\$2M, compressed into a 140GB file — roughly 100x compression. Karpathy calls it “a zip file of the internet,” but lossy: the model doesn't store the text, it stores a generalized, gestalt-like representation. Frontier models at ChatGPT/Claude scale are an order of magnitude beyond this — tens to hundreds of millions of dollars.
- **The core task is next-word prediction.** From “Cat sat on a,” predict the next token (e.g., “mat,” ~97% probability). This objective, applied at internet scale, is what forces the model to absorb enormous amounts of world knowledge as a side effect of getting good at prediction.
- **“Dreaming,” not recalling.** Inference is the model repeatedly predicting the next token and feeding it back in — Karpathy calls this “dreaming internet documents.” Output is often plausible but hallucinated: fabricated ISBNs, invented facts about obscure topics. The model is not looking anything up; it is generating what's statistically likely to come next.
- **Inscrutable by design.** Knowledge is distributed across billions of parameters in ways we don't understand even though we know how to train them. The “reversal curse” example: a model that knows “Tom Cruise's mother is Mary Lee Pfeiffer” often fails to answer “who is Mary Lee Pfeiffer's son” — the same fact doesn't generalize across direction. Karpathy's framing: these are “mostly inscrutable artifacts,” evaluated empirically like a natural phenomenon, not engineered like a bridge.

How a base model becomes an assistant

- **Stage 1 — Pretraining.** Massive, low-quality-per-token internet data → a base model that only continues documents; it cannot follow instructions. Expensive (millions of dollars), infrequent.
- **Stage 2 — Fine-tuning (supervised fine-tuning, SFT).** Retrain on a small, high-quality, human-curated set of Q&A conversations (order of 100,000 examples). This is what turns a document-completer into something that responds helpfully to

“can you help me with this code?” Far cheaper, done far more often.

- **Stage 3 — RLHF (optional but standard).** Humans find it easier to *rank* several candidate answers than to *write* a perfect one, so labelers compare outputs (e.g., rank several haiku attempts) and those comparisons train the model further via Reinforcement Learning from Human Feedback. Labeling instructions (from the InstructGPT paper: “helpful, truthful, harmless”) run to tens or hundreds of pages — the apparent simplicity of a good chatbot response hides a large annotation operation.
- **The critical caveat: fine-tuning doesn't fix hallucination.** It redirects the model's “dreaming” toward helpful-assistant-shaped dreams. It does not make the model factually grounded. Treat model output as unverified unless it came from retrieval/browsing you can check — the same caution Mollick raises in Ch. 1 (plausibility ≠ accuracy).
- **Evaluation is relative, not absolute.** Models are ranked by Elo (borrowed from chess) on the Chatbot Arena leaderboard — head-to-head human preference votes, not a fixed test score. At the time of this talk, closed models (OpenAI, Anthropic) led; open-weight models (Llama, Mixtral) trailed but were closing. (Reading 2 gives you where that gap stands in 2026.)

Where the field was heading (from a Nov 2023 vantage point)

- **Scaling laws.** Performance is a predictable function of parameter count (N) and training data volume (D) — more of both reliably helps, independent of algorithmic cleverness. This predictability is what justified the capital buildout you now see in the AI Index's Economy chapter.
- **Tool use.** The model emits a special token when it needs a tool (browser, calculator, Python interpreter, image generation); surrounding code intercepts that token, runs the tool, and feeds the result back into the context. It doesn't “decide” to use tools by magic — it learned when to emit the trigger token from fine-tuning examples. This is the exact mechanism your Class 4 agent will use.
- **Multimodality.** From a hand-sketched UI mockup to working HTML/JS; speech-to-speech interaction. Text was never the ceiling, just the easiest modality to start with.
- **System 1 vs. System 2 (Kahneman).** Base LLM inference is pure System 1 — fast, reflexive, one token at a time, no deliberation. Karpathy frames the open research problem as building System 2: trading compute time for answer quality (tree-of-thought and similar techniques were the then-current attempts). This is precisely the problem that today's “reasoning models” (extended thinking, chain-of-thought at inference time) are built to solve — the field's answer to a question this talk left open.
- **Self-improvement.** AlphaGo's arc — imitate humans, then improve beyond them via self-play — is the aspiration for LLMs. The blocker Karpathy names: Go has a crisp win/loss reward signal; most language tasks don't. Progress here is narrow-domain only.
- **LLM as operating system.** Not a chatbot — a kernel coordinating resources: context window as RAM, embeddings/retrieval as file system, tools as installed software, other models as coprocessors. Proprietary ecosystems (GPT, Claude) versus an open one building around Llama, mirroring closed vs. open desktop OSes. This metaphor is still the standard way agent architectures are described in 2026.

Security — a new attack surface

- **Jailbreaks.** Safety training is bypassed by role-play framing (the “grandmother used to tell bedtime stories about how to make napalm” trick), by base64-encoding the harmful request (because safety tuning concentrates on plain English), by appending an adversarially-optimized “universal transferable suffix,” and by hiding instructions in crafted image noise. Different mechanism, same result: the safety layer is a thin, learned behavior sitting on top of a much less constrained base capability.
- **Prompt injection.** Instructions hidden where a human won't notice them but the model will read them — near-invisible text in an image, a compromised webpage a search tool visits, malicious content in a shared document combined with a model's own ability to render URLs or run scripts (used to exfiltrate data). The common thread: anything the model reads is untrusted input, not just what the user typed.
- **Data poisoning / backdoors.** A trigger phrase inserted during fine-tuning data causes reliably malicious behavior only when that phrase later appears (Karpathy's example: a “James Bond” trigger). Demonstrated at fine-tuning scale; the same idea extends in principle to pretraining data.
- **The verdict: a permanent cat-and-mouse game,** structurally identical to traditional computer security — every patch narrows one exploit while the underlying attack surface (a system that reads and acts on arbitrary text) remains open. Directly relevant to your A2 guardrail checklist.

READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these are meant to surface how you think, not to test recall.

1. Karpathy says fine-tuning only “redirects the dreams” of an LLM toward being a helpful assistant — it doesn't stop the hallucinating. If you ask an LLM a question and it answers confidently with no sources or links attached, how much should you trust that answer? What would make you trust it more?
2. This talk is from November 2023 — around when most of you were finishing high school or partway through university. Think of something in tech — an app, a gadget, a way of doing something — that felt cutting-edge to you back then and now feels outdated. What changed, and what tipped you off that it had?
3. The LLM-OS metaphor treats the context window as RAM and retrieval/browsing as a file system — the difference between “what's loaded and ready right now” and “what's stored somewhere and has to be fetched.” You've lived that distinction on your own phone or laptop — an app slowing down when it's overloaded, or needing to “look something up” instead of already knowing it. Where have you personally run into that? How is it similar to, or different from, what an LLM does?

SOURCE LINKS

Video · [YouTube — Intro to Large Language Models](#)

run.c reference code · github.com/karpathy/llama2.c

Companion reading (optional) · [The Unreasonable Effectiveness of Recurrent Neural Networks](#)

CAVEATS

1. **This summary is not a substitute for the reading.** It is a map, not the territory. Watch the talk — Karpathy's live demonstrations (the model “dreaming” text, the jailbreak examples) land differently on video than in a bulleted summary.
2. **This summary was generated and fact-checked using Anthropic models**, cross-referenced against two independent published summaries and the original 2023 link post confirming publication date and timestamps. Errors are still possible — the talk itself is the authority. Flag anything that looks wrong.